

NIST AI RMF Profile

Self-Assessment Against AI RMF 1.0 (NIST AI 100-1)

Version	1.0
Date	2026-05-06
Author	Jay Wingard
Subject System	Genesis Autonomous AI Agent
Classification	Public

Document History

Version / Date	Author	Description
1.0 (2026-05-06)	Jay Wingard	Initial release

Contents

1	NIST AI RMF Profile: Genesis Autonomous AI Agent	3
1.1	1. Executive Summary	3
1.1.1	1.1.1 Risk Tolerance Statement	3
1.1.2	1.1.2 Trustworthiness Characteristics Summary	3
1.2	2. GOVERN Function Assessment	4
1.2.1	1.2.1 GOVERN 1: Policies, processes, procedures, and practices . . .	4
1.2.2	1.2.2 GOVERN 2: Accountability structures	6
1.2.3	1.2.3 GOVERN 3: Workforce diversity, equity, inclusion, and acces- sibility	6
1.2.4	1.2.4 GOVERN 4: Culture of risk communication	7
1.2.5	1.2.5 GOVERN 5: Engagement with relevant AI actors	7
1.2.6	1.2.6 GOVERN 6: Third-party software, data, and supply chain (Deep Analysis)	8
1.3	3. MAP Function Assessment	9
1.3.1	1.3.1 MAP 1: Context is established and understood	9
1.3.2	1.3.2 MAP 2: Categorization of the AI system	9
1.3.3	1.3.3 MAP 3: Capabilities, usage, goals, benefits, and costs	10
1.3.4	1.3.4 MAP 4: Third-party risks	11
1.3.5	1.3.5 MAP 5: Impact characterization	11
1.4	4. MEASURE Function Assessment	12
1.4.1	1.4.1 MEASURE 1: Methods and metrics	12
1.4.2	1.4.2 MEASURE 2: Trustworthiness evaluation	12
1.4.3	1.4.3 MEASURE 3: Risk tracking over time	15
1.5	5. MANAGE Function Assessment	15
1.5.1	1.5.1 MANAGE 1: Risk prioritization and response	15
1.5.2	1.5.2 MANAGE 2: Strategies for benefits and impact management .	15
1.5.3	1.5.3 MANAGE 3: Third-party risk management	16
1.5.4	1.5.4 MANAGE 4: Risk treatment documentation and monitoring . .	17
1.6	6. Gap Summary and Prioritization	17
1.6.1	1.6.1 Critical Gaps	17
1.6.2	1.6.2 Significant Gaps	18
1.6.3	1.6.3 Minor Gaps	18
1.6.4	1.6.4 Framework Adaptation Notes	18

1.7	7. Remediation Roadmap	19
1.8	Appendix A: Operational Evidence	19
1.8.1	System Scale	19
1.8.2	Approval Gate Evidence	20
1.8.3	Autonomy State Evidence	20
1.8.4	Ego System Evidence	21
1.8.5	Event Distribution	21
1.8.6	Memory Distribution	21

NIST AI RMF Profile: Genesis Autonomous AI Agent

Version: 1.0 | Date: 2026-05-06 | Classification: Public

1. Executive Summary

This document presents a self-assessment of the Genesis autonomous AI agent system against the NIST Artificial Intelligence Risk Management Framework (AI RMF 1.0, NIST AI 100-1, January 2023). The assessment follows the Current Profile / Target Profile / Gap Analysis structure described in Section 6 of the AI RMF.

Genesis is a single-tenant autonomous agent operating on behalf of one human user. It takes external actions (sending messages, executing code, browsing the web), maintains persistent memory, and operates proactively during periods without direct user oversight. These characteristics distinguish it from the classification systems, recommender engines, and bounded chatbots that inform most of the AI RMF's examples and assumptions.

The assessment covers all four AI RMF functions (GOVERN, MAP, MEASURE, MANAGE) across their 24 categories and 72 subcategories. Rather than providing uniform shallow coverage, this profile allocates depth according to governance relevance: subcategories where Genesis presents unusual or elevated risk receive detailed analysis, while subcategories that map straightforwardly to existing controls receive brief treatment. Seven subcategories receive deep analysis based on their relevance to autonomous agent governance.

The system description (D0) provides the factual foundation for this assessment and should be read first.

Risk Tolerance Statement

Genesis operates within a personal-use, single-operator risk context. The operator accepts elevated risk in exchange for the system's autonomous capabilities, bounded by the invariant that irreversible actions require explicit human approval with no timeout. Risk tolerance is higher for internal actions (memory operations, code modifications within the container) than for external actions (communications to other people, financial transactions, public-facing operations). The operator's risk tolerance reflects a development and personal-productivity context rather than enterprise, healthcare, or safety-critical deployment.

Trustworthiness Characteristics Summary

The AI RMF identifies seven characteristics of trustworthy AI systems. Their applicability to Genesis:

Valid and Reliable: Partially applicable. Genesis does not produce predictions in the traditional sense. Its outputs are actions, communications, and synthesized

knowledge. Validity is assessed through correction rates and the Bayesian trust model rather than statistical accuracy metrics.

Safe: Applicable with adaptation. Safety in the Genesis context means preventing the system from taking harmful external actions, not preventing physical harm. The approval gate architecture is the primary safety mechanism.

Secure and Resilient: Applicable. The system faces standard cybersecurity risks plus AI-specific risks including prompt injection and adversarial manipulation of advisory-layer constraints. Provider circuit breakers provide resilience through and the Guardian/Sentinel monitoring architecture.

Accountable and Transparent: Applicable and well-addressed. The approval lifecycle, event bus, session transcripts, and Git history provide a thorough audit surface. The single-operator model simplifies accountability (one person is responsible for all decisions about the system).

Explainable and Interpretable: Partially applicable. LLM-based reasoning is inherently difficult to explain at the mechanism level. The system compensates through confidence labeling, evidence citation, and structured proposal formats that surface the reasoning basis for autonomous decisions.

Privacy-Enhanced: Applicable with context. The single-user model means the primary privacy concern is data transmitted to third-party LLM providers, not multi-user data separation. Secondary concerns involve personal data of external individuals stored in the system's memory.

Fair-with Harmful Bias Managed: Limited applicability. Genesis is designed to serve one user's interests. It does not make decisions about populations or allocate resources across groups. Bias considerations apply to how the system prioritizes information and selects outreach targets, not to the traditional fairness concerns of classification systems.

2. GOVERN Function Assessment

The GOVERN function establishes organizational risk management culture, structures, and processes. In the Genesis context, "organization" maps to a single operator managing a single system. Many GOVERN subcategories assume institutional structures (boards, committees, workforce diversity programs) that do not apply to a solo-operated system. Where this is the case, the assessment documents the inapplicability and identifies what, if anything, serves an equivalent function.

GOVERN 1: Policies, processes, procedures, and practices

GOV-1.1: Legal and regulatory requirements involving AI are understood, managed, and documented.

Current State: Partial. The system's deployment context (personal use, single operator, US-based) reduces but does not eliminate legal exposure. EU AI Act applicability analysis is documented separately (D3). No formal legal register exists.

Target State: Document applicable regulatory requirements (EU AI Act transparency obligations for AI-generated content, GDPR considerations for data transmitted to EU-based processors, terms of service compliance for LLM providers). Maintain as a living reference.

Gap: No formal legal requirements register. The EU AI Act assessment (D3) partially fills this gap but is a point-in-time analysis rather than an ongoing monitoring process.

GOV-1.2: The characteristics of trustworthy AI are integrated into organizational policies, processes, procedures, and practices.

Current State: Implemented through design. Trustworthiness characteristics are embedded in the system architecture rather than in policy documents: the approval gate enforces safety, the event bus enables accountability, the correction mechanism addresses reliability, and the defense-in-depth model addresses security. Steering rules (user-defined behavioral constraints) function as the closest analog to organizational policy.

Target State: Current implementation is appropriate for the deployment context. Formalizing these architectural decisions as explicit policies would add documentation overhead without changing actual behavior.

Gap: Minor. Architectural implementation exceeds what a policy document would specify, but the absence of a unified policy document means the relationship between trustworthiness characteristics and system controls is not documented in one place. This system description partially addresses that gap.

GOV-1.3: Processes, procedures, and practices are in place to determine the needed level of risk management activities based on the organization's risk tolerance.

Current State: Implemented. The action classification system (REVERSIBLE / COSTLY_REVERSIBLE / IRREVERSIBLE) directly maps risk tolerance to management activities. Higher-risk actions receive more management (approval gates, longer timeouts, human-in-the-loop requirements). The risk tolerance statement in this document formalizes what was previously an implicit understanding.

Target State: Current implementation is appropriate.

Gap: None identified.

GOV-1.4 through GOV-1.5: Transparent policies and periodic review.

Current State: Partial. System behavior is transparent to the operator (full database access, session transcripts, event logs). No formal periodic review schedule exists; review occurs reactively in response to incidents or behavioral drift.

Target State: Establish a periodic review cadence for system governance controls, even if informal (quarterly review of steering rules, autonomy levels, and correction patterns).

Gap: No scheduled review process. Risk management review is reactive rather than periodic.

GOV-1.6: Mechanisms are in place to inventory AI systems.

Not applicable in the traditional sense—there is one system. The bootstrap manifest tracks subsystem capabilities and operational status at the component level.

GOV-1.7: Processes for decommissioning and phasing out AI systems.

The system can be terminated (stop the systemd service, destroy the container) and data can be deleted (drop the database, clear the file system). What is missing is a procedural checklist for external cleanup: revoking API keys, requesting data deletion from LLM providers, notifying outreach recipients where appropriate. The technical shutdown capability exists; the procedural discipline does not.

GOVERN 2: Accountability structures**GOV-2.1: Roles and responsibilities are documented and clear.**

Current State: Implemented, though the structure is unusual. In a single-operator system, all roles collapse to one person: developer, deployer, operator, and data controller. Within the system, responsibilities are distributed between the human (strategic direction, approval decisions, behavioral constraints) and Genesis (execution, monitoring, proposal generation). This division is documented in the system description (D0, Section 3) and enforced through the autonomy model.

Target State: Current implementation is appropriate for the deployment context.

Gap: None identified.

GOV-2.2: Personnel receive AI risk management training.

Not applicable. The single operator built the governance controls, which is a stronger competence signal than any training program.

GOV-2.3: Executive leadership takes responsibility for AI risk decisions.

Implemented. The single operator makes all risk-bearing decisions. The no-auto-promote invariant and the irreversible-action-waits-forever rule prevent the operator from delegating risk acceptance to the system by neglect. Operational evidence: 107 approval requests processed, with the operator engaging through multiple resolution channels (Telegram, dashboard)—see Appendix A.

GOVERN 3: Workforce diversity, equity, inclusion, and accessibility**GOV-3.1: Decision-making is informed by a diverse team.**

Current State: Not applicable. Single-operator system. The operator's perspective is the only human perspective in the system's governance.

This is an honest limitation, however, not one that can be addressed architecturally. The AI RMF's emphasis on diverse perspectives reflects the reality that individual blind spots create systemic risk. A solo-operated autonomous agent inherits its operator's biases, priorities, and blind spots without the corrective influence of a diverse team. The system's correction mechanism provides a partial mitigation: when the operator notices problematic behavior and corrects it, the system adapts. But the operator cannot correct what they do not notice.

GOV-3.2: Policies and procedures define roles for human-AI configurations and oversight.

Current State: Implemented. The autonomy model (D0, Section 4) defines four levels of human-AI interaction, from fully supervised (L1) to proactive with approval

gates (L4). The division of responsibility between human oversight and system autonomy is the central design concern of the system's governance architecture.

GOVERN 4: Culture of risk communication

GOV-4.1: Safety-first mindset in design, development, deployment, and use.

Current State: Implemented. The system's design reflects a safety-biased posture: trust accumulates slowly, regresses quickly; irreversible actions wait indefinitely; timeout means rejection. The steering rules mechanism allows the operator to inject behavioral constraints in response to observed risks.

GOV-4.2: Teams document risks and communicate about impacts.

Current State: Partial. Risks are documented in this assessment series and in the system description. The event bus and alert system communicate about impacts in real time. No periodic risk communication process exists beyond reactive alerting.

GOV-4.3: Practices for AI testing, incident identification, and information sharing.

Current State: Implemented. The system includes a test suite, the Guardian subsystem performs continuous health monitoring, the Sentinel classifies and responds to incidents, and the event bus provides structured information sharing. Incident identification is automated; information sharing is internal to the operator.

GOVERN 5: Engagement with relevant AI actors

GOV-5.1: Policies for collecting and integrating external feedback.

Current State: Partial. The open-source nature of the Genesis codebase allows external review, but no formal mechanism exists for collecting feedback from external parties affected by the system's actions (outreach recipients, people referenced in memory). The correction mechanism captures feedback from the operator only.

Target State: For a personal-use system, formal external feedback mechanisms are disproportionate to the risk. If the system were deployed in a context affecting more people, this would require a feedback channel accessible to affected parties.

Gap: No external feedback mechanism. Appropriate for current deployment context; would require attention if deployment context changed.

GOV-5.2: Mechanisms for incorporating adjudicated feedback.

Current State: Implemented for operator feedback. The correction-to-regression pipeline (D0, Section 7.4) translates operator feedback into system behavioral changes with quantitative tracking.

GOVERN 6: Third-party software, data, and supply chain (Deep Analysis)**GOV-6.1: Policies and procedures address AI risks associated with third-party entities.**

Current State: Partial. The system's dependence on third-party LLM providers is its most significant supply chain risk. Genesis relies on Anthropic Claude for primary reasoning, with secondary routing to DeepInfra, Groq, and Mistral. Over 58 days of operation, 2,591 cost events have been recorded across these providers. The system transmits operational context (including memory excerpts and conversation history) to these providers as part of normal operation.

Current mitigations are technical, not procedural: - Circuit breaker pattern (`src/genesis/router`) provides failover between providers - Cost tracking monitors per-provider token usage (2,591 tracked cost events) - The routing system can redirect traffic away from degraded providers - Secrets management (`secrets.env`, `chmod 600`) isolates API credentials

What is missing: - No formal assessment of provider data retention and processing practices - No contractual protections specific to the sensitivity of data transmitted - No mechanism to verify provider compliance with their stated data handling policies - No impact assessment of what would happen if a provider changed its terms of service, data handling practices, or model behavior - No assessment of intellectual property risks (prompts containing proprietary information transmitted to providers)

Target State: Document provider data handling practices and terms of service relevant to Genesis operations. Assess the sensitivity of data transmitted to each provider. Establish a monitoring cadence for provider policy changes.

Gap: Significant. Third-party LLM dependency is the system's most significant governance gap. The system depends on external providers for all reasoning capability and transmits sensitive operational data to those providers without formal governance of that data flow. This gap is partially structural (the system cannot function without LLM providers) and partially procedural (no documentation or monitoring of provider practices).

GOV-6.2: Contingency processes for failures in third-party data or AI systems.

Current State: Partial. The circuit breaker pattern provides automatic failover between providers. The dead-letter queue captures failed requests. The Guardian monitors provider health as part of regular health checks.

What is missing: No contingency plan for a sustained outage of all LLM providers simultaneously. No plan for responding to a provider data breach that includes Genesis operational data. No assessment of how model behavior changes (intentional or unintentional) across provider versions might affect system governance properties.

In practice, technical resilience is addressed (circuit breakers), but operational contingency planning for worst-case scenarios is not documented.

3. MAP Function Assessment

The MAP function establishes context for risk identification. Many MAP subcategories are addressed in the system description (D0) and are cross-referenced here rather than repeated.

MAP 1: Context is established and understood

MAP-1.1 (intended purposes and impacts): Implemented. Documented in D0.

MAP-1.2 through MAP-1.4 (interdisciplinary actors, mission, business value): MAP-1.2 has limited applicability (single operator). MAP-1.3 and MAP-1.4 are implicit in the system's design intent rather than formally documented as business objectives.

MAP-1.5 (risk tolerances): Implemented—see Section 1 of this document.

MAP-1.6 (system requirements and socio-technical implications): Partial. Requirements are expressed in the system design rather than in a formal specification. Socio-technical implications are addressed through the approval gate architecture (managing social consequences of autonomous action) and the outreach pipeline (managing the social impact of automated communications).

MAP 2: Categorization of the AI system

MAP-2.1 (tasks and methods): Implemented. The system description identifies tasks (perception, cognition, action) and methods (LLM inference, Bayesian trust modeling, structured event processing).

MAP-2.2: Knowledge limits and human oversight are documented (Deep Analysis).

Current State: Partial. The system's knowledge limits are partially documented:

Documented limits: - The system acknowledges confabulation risk in its operational context (steering rules explicitly require labeling speculation and acknowledging uncertainty) - The confidence labeling requirement forces the system to indicate when it is uncertain - Memory verification procedures require the system to check recalled information against current state before acting on it

Undocumented or poorly bounded limits: - No formal specification of what the system should and should not attempt to do autonomously. The autonomy model constrains how the system acts (approval gates), but not the scope of what it attempts within those constraints. - No defined boundary between tasks the system is competent to handle and tasks it should defer to the user. The system's self-assessment of competence depends on LLM judgment, which is unreliable for precisely the cases where it matters most (unknown unknowns). - The system can encounter novel situations where no correction history provides guidance. In these cases, the Bayesian trust model defaults to caution (L1), but the system may still take reversible actions that reflect poor judgment.

Target State: Document explicit knowledge limits: categories of tasks the system should not attempt regardless of autonomy level, indicators that should trigger automatic deferral to the user, and boundary conditions for the system's self-

assessment of competence.

This is a general challenge for LLM-based agents, not specific to Genesis. The model's uncertainty about its own competence boundary is itself unreliable—the cases where the system most needs to recognize its limits are precisely the cases where it is least equipped to do so. Traditional software either handles an input or throws an error. An LLM-based agent will attempt to handle anything, with varying quality, and no reliable internal signal distinguishes “handled well” from “handled confidently but poorly.”

Gap: The system constrains its authority (what it can do) but not its judgment about competence (what it should attempt).

MAP-2.3: Scientific integrity and TEVV considerations.

Current State: Partial. The system includes a test suite and the correction mechanism provides a form of ongoing validation. No formal TEVV process exists beyond testing and the behavioral correction loop.

MAP 3: Capabilities, usage, goals, benefits, and costs

MAP-3.1 through MAP-3.3 (benefits, costs, application scope): Benefits and costs are understood informally. No formal cost-benefit analysis exists. Application scope is bounded by the container environment and configured integrations.

MAP-3.4 (operator proficiency): Implemented. The operator designed and built the system.

MAP-3.5: Processes for human oversight are defined, assessed, and documented (Deep Analysis).

Current State: Implemented. Human oversight is the central design concern of the Genesis governance architecture, documented in the system description (D0, Section 4) and assessed here.

The oversight model operates at multiple levels:

1. **Continuous oversight** (always active): The operator can observe system behavior through session transcripts, event logs, and the dashboard. All actions and their outcomes are logged.
2. **Approval oversight** (triggered by action classification): Actions classified as costly-reversible or irreversible require explicit human approval before execution. This is enforced in `src/genesis/autonomy/approval_gate.py`, not through advisory constraints. Operational evidence: 107 approval requests have been processed, with the operator engaging through Telegram (78.5%) and dashboard (11%)-indicating active, multi-channel oversight rather than rubber-stamping.
3. **Autonomy ceiling oversight** (structural): The system cannot exceed its assigned autonomy level (`src/genesis/autonomy/state_machine.py`). Level advancement requires explicit user action. The background cognitive category has reached L4 with 877 successes against 4 corrections (posterior 0.998), while outreach remains at L1 with no evidence—demonstrating that autonomy expands only where evidence accumulates.
4. **Correction oversight** (reactive): When the operator corrects system behavior, the correction feeds into the Bayesian trust model, potentially regressing the system's autonomy level. However, the trust formula has a small-sample limi-

tation: the `direct_session` category reached L4 with only 5 successes and 1 correction (posterior 0.75). At this sample size, the Laplace smoothing dominates and the posterior conveys limited statistical confidence. The system reached its highest autonomy level on thin evidence—a known design tradeoff favoring operational flexibility over statistical rigor at small scale.

5. **Emergency oversight:** The operator can pause, restart, or terminate the system at any time. The Sentinel’s alarm response requires human approval before remediation actions are taken.

What the AI RMF’s human oversight framework does not address well:

The AI RMF assumes human oversight means a human reviews AI outputs before they affect the world. Genesis operates in a model where some actions are pre-authorized (reversible actions at L2+) and the human only reviews a subset. The system’s safety depends on the accuracy of the action classification system. If an action is misclassified as REVERSIBLE when it should be COSTLY_REVERSIBLE or IRREVERSIBLE, it bypasses the approval gate entirely.

The classification system uses keyword pattern matching with LLM override capability. This is a known weakness: novel action types that don’t match existing patterns default to REVERSIBLE. The system is designed to be safe against this specific failure mode for the most serious cases (financial transactions, destructive operations have explicit keyword patterns), but the long tail of potentially harmful actions that are technically reversible but socially costly is not fully covered.

Target State: Current implementation is appropriate for the deployment context, with the caveat that action classification accuracy should be monitored over time.

Gap: Action classification accuracy is not formally measured. The correction mechanism provides an indirect signal (corrections indicate classification failures), but no direct metric tracks the false-negative rate of the classification system.

MAP 4: Third-party risks

Addressed under GOV-6 above.

MAP 5: Impact characterization

MAP-5.1: Likelihood and magnitude of impacts are identified and documented.

Current State: Partial. The system description (D0, Sections 5 and 8) identifies the categories of external impact and known limitations. No formal likelihood estimation or magnitude assessment has been performed.

MAP-5.2: Regular engagement with relevant AI actors.

Current State: Limited. No formal engagement process with people affected by the system’s actions.

4. MEASURE Function Assessment

MEASURE 1: Methods and metrics

MEASURE-1.1: Approaches and metrics for measurement of AI risks are selected.

Current State: Partial. The Bayesian trust model provides a quantitative metric for behavioral reliability—current posteriors range from 0.998 (background_cognitive, 881 observations) to undefined (outreach, 0 observations). Provider health metrics (response time, error rate, circuit breaker state) measure operational risk across 2,591 tracked cost events. The event bus captures 135,843 events across 10+ subsystems, with severity distribution providing a coarse system health signal (91.2% debug, 4.5% error, 3.0% warning, 1.3% info).

What is not measured: action classification accuracy (all 107 approval requests were COSTLY_REVERSIBLE—either the system never encounters IRREVERSIBLE actions, or the classifier never flags them; the 0% IRREVERSIBLE rate itself warrants investigation), confabulation rate (how often the system acts on incorrect recalled information out of 12,164 stored memories), outreach quality (12,800 outreach events with no outcome tracking).

Target State: Add tracking for action classification outcomes (did the action actually have the consequences its classification predicted) and confabulation incidents (cases where memory verification reveals a discrepancy between recalled and actual state).

Gap: Measurable metrics exist for operational health and behavioral reliability but not for the AI-specific risks most relevant to an autonomous agent (classification accuracy, confabulation rate, judgment quality).

MEASURE-1.2 through MEASURE-1.3 (regular assessment, independent review): MEASURE-1.2 is not implemented—no schedule for assessing metric effectiveness. MEASURE-1.3 has limited applicability (single operator, no independent assessors).

MEASURE 2: Trustworthiness evaluation

MEASURE-2.1 through MEASURE-2.3: TEVV documentation and deployment testing.

Current State: The system has a test suite covering functional correctness. No formal TEVV process covers trustworthiness characteristics specifically. Performance is assessed in deployed conditions (the system is tested in the same environment it operates in).

MEASURE-2.4: Production monitoring of functionality and behavior (Deep Analysis).

Current State: Implemented, with gaps.

What is monitored in production (with representative figures from 58 days of operation): - Subsystem health via Guardian (src/genesis/guardian/) and Sentinel (src/genesis/sentinel/): tick regularity, memory pressure, error rates, restart counts, pause state, autonomous alarm classification and dispatch - Provider per-

formance: response time, error rate, circuit breaker state per provider (9,772 routing events logged) - Event bus activity: 135,843 events across subsystems; dashboard alone generated 74,597 events, awareness 17,233, surplus 15,600 - Autonomy state: current/earned levels, success/correction counts per category (877/4 for background_cognitive, 5/1 for direct_session) - Approval lifecycle: 107 requests processed, median resolution via Telegram batch (54.2%) - Budget consumption: token usage per model, per ego cycle (26 cycles, 28 proposals generated)

What is not monitored: - Behavioral drift: no metric tracks whether the system's decision patterns are changing over time in ways that might indicate degradation or misalignment. The correction mechanism catches cases the operator notices, but systematic drift that stays below the operator's attention threshold could accumulate undetected. - Output quality: no automated assessment of whether the system's outputs (communications, research results, synthesized knowledge) meet quality standards. Quality assessment depends entirely on operator review, which is sampled rather than exhaustive. - Memory integrity: no ongoing verification that stored memories remain accurate as the world changes. A fact stored six months ago may be wrong today, and the system may act on it without checking.

This is a structural challenge for autonomous agents that existing monitoring practices do not address well. Traditional AI monitoring assumes a measurable output (predictions, classifications) that can be compared to ground truth. Genesis produces a heterogeneous mix of actions, communications, and knowledge artifacts with no single ground truth to compare against.

Target State: Implement behavioral drift detection (track correction rate trends over time, flag increasing correction rates in specific categories). Add memory staleness detection (flag memories that have not been verified against current state within a configurable window).

Gap: Operational monitoring is well-implemented. Behavioral monitoring is a significant gap, particularly for an autonomous system that can accumulate subtle behavioral drift over time.

MEASURE-2.5 through MEASURE-2.10: Validity, safety, security, transparency, explainability, privacy.

These subcategories are addressed in the trustworthiness characteristics summary (Section 1) and the system description. Key points:

- Validity (2.5): Assessed through correction rates rather than statistical accuracy. Limitations of generalizability are documented (the system performs well in its operator's domains and poorly in unfamiliar domains, limited by the operator's ability to detect errors).
- Safety (2.6): Addressed through the approval gate architecture. The system can fail safely: if the approval mechanism fails, actions requiring approval do not execute (fail-closed design for irreversible actions).
- Security (2.7): Standard cybersecurity measures plus AI-specific mitigations (defense-in-depth against prompt injection, protected paths against unauthorized file modification). Not formally evaluated against an adversarial threat model.
- Transparency (2.8): Well-addressed. Full audit trail, session transcripts, operator access to all system state.
- Explainability (2.9): Partially addressed. The system provides confidence labels

and evidence citations but cannot explain the internal mechanism of LLM-based reasoning. Proposal formats make the reasoning basis for autonomous decisions visible, but “why did the model produce this output” remains opaque.

- Privacy (2.10): Addressed in the system description (D0, Section 6). The primary privacy risk is data transmitted to LLM providers.

MEASURE-2.11: Fairness and bias are evaluated (Deep Analysis).

Current State: Limited, but this requires careful framing.

The AI RMF’s fairness and bias framework is designed for systems that make decisions about populations: credit scoring, hiring, criminal justice, healthcare allocation. Genesis does not make decisions about populations. It makes decisions about how to serve one user’s interests.

Where bias is relevant to Genesis: - **Information prioritization bias:** The system’s memory retrieval and signal triage may systematically surface some types of information over others, creating a filter bubble effect for the operator. If the system consistently prioritizes certain news sources, perspectives, or contact categories, it could narrow the operator’s awareness without their knowledge. - **Outreach target selection:** When the system selects targets for proactive communication, its selection criteria may embed biases from training data or from patterns in the operator’s past behavior. - **LLM-inherited biases:** The underlying language models carry biases from their training data. When Genesis uses LLM judgment for classification, prioritization, or content generation, those biases propagate into system behavior.

Where bias is not meaningfully applicable: - Demographic fairness across populations (the system serves one person) - Equal treatment of applicants or candidates (the system does not evaluate people for opportunities) - Protected-class discrimination (the system’s actions are directed by one person’s interests, not institutional policies)

Current mitigations: - The operator reviews and corrects system outputs, providing a human check on biased behavior - The correction mechanism penalizes biased behavior when detected - No formal bias assessment or monitoring exists

Target State: Monitor information diversity in memory retrieval and outreach targeting. Track whether the system’s attention distribution shifts over time in ways that narrow the operator’s information diet.

The AI RMF’s bias framework was built for a different problem. Adapting it for single-user agentic systems is not a matter of applying the same assessment at smaller scale—it requires rethinking what bias means when the system serves one person rather than deciding about many.

Gap: No formal bias assessment or monitoring exists for the types of bias actually relevant here (information prioritization, attention narrowing, LLM-inherited patterns).

MEASURE-2.12: Environmental impact.

Current State: Not assessed. The system’s compute footprint is modest (single container, API-based inference), but the total energy cost including LLM provider infrastructure is not tracked.

MEASURE-2.13: Effectiveness of TEVV metrics.

Current State: Not implemented. No meta-assessment of measurement effectiveness.

MEASURE 3: Risk tracking over time

MEASURE-3.1 through MEASURE-3.3 (ongoing risk tracking and feedback): Partial. Correction mechanism tracks behavioral risk; Guardian tracks operational risk. No mechanism tracks emergent risks outside these two systems, and no feedback process exists for affected external parties.

MEASURE-4.1 through MEASURE-4.3 (measurement efficacy feedback): Not implemented. No meta-assessment of whether existing measurements produce useful risk information.

5. MANAGE Function Assessment

MANAGE 1: Risk prioritization and response

MANAGE-1.1 (achieving intended purposes): Implicit—the operator uses the system daily and has direct visibility. No formal evaluation criteria documented.

MANAGE-1.2 (risk treatment prioritization): Partial. The action classification system implicitly prioritizes (irreversible actions get the strongest controls). No formal risk register exists beyond this assessment series.

MANAGE-1.3 (high-priority risk responses): Partial. Approval gates handle high-priority action risks. The Sentinel handles system health risks. No documented response plan for risks outside these categories—provider data breaches, regulatory action, sustained multi-provider outage.

MANAGE-1.4 (residual risk documentation): Implemented in this assessment. Primary residual risks: third-party LLM dependency, advisory-layer constraint bypass, behavioral drift below operator attention threshold, confabulation risk.

MANAGE 2: Strategies for benefits and impact management

MANAGE-2.1: Resources for risk management, including non-AI alternatives.

The non-AI alternative is always available—the operator can perform any task manually because the system augments rather than replaces.

MANAGE-2.2 (sustaining deployed value): Implemented. The learning pipeline (6,175 observations, 12,164 memories) continuously adapts the system. Regular codebase updates add capabilities.

MANAGE-2.3 (responding to unknown risks): Partial. Corrections handle unknown behavioral risks; the Sentinel handles unknown operational risks. No procedure addresses reputational, legal, or regulatory risks.

MANAGE-2.4: Mechanisms to supersede, disengage, or deactivate (Deep Analysis).

Current State: Implemented at multiple levels.

Immediate disengagement: The operator can stop the systemd service at any time, immediately halting all system activity. This is the kill switch. It requires no cooperation from the system and no approval process.

Selective disengagement: The operator can pause specific subsystems (ego, outreach, surplus) without stopping the entire system. This allows targeted risk reduction without losing monitoring and memory capabilities.

Approval rejection as disengagement: Rejecting an approval request prevents the specific action from executing. The system records the rejection, updates the Bayesian trust model, and does not retry the rejected action. For irreversible actions, rejection is permanent. In practice, 3 of 107 approval requests expired without response (2.8%) and 7 were cancelled (6.5%), but none were explicitly rejected-suggesting the operator has not yet needed to use rejection as a disengagement tool.

Autonomy regression as graduated disengagement: The operator can demote the system's autonomy level, reducing the scope of actions it can take without approval. The ego system provides evidence that proactive governance is working: of 28 proposals generated, 3 were rejected (10.7%) and 11 were withdrawn (39.3%), showing that the proposal-review cycle filters proactive actions before they reach the approval gate.

Sentinel-triggered disengagement: The Sentinel can propose system-level responses to health alarms, including pausing subsystems. These proposals require operator approval before execution.

What works well: The disengagement mechanisms are layered, providing graduated responses from full shutdown to targeted subsystem pausing. The kill switch (stopping the systemd service) operates independently of the system's own governance layer.

What is missing: No automated disengagement trigger that operates without human involvement. If the system is behaving dangerously and the operator is not available to intervene, no mechanism automatically constrains it. The irreversible-action-waits-forever invariant partially addresses this (the system cannot take irreversible actions without the operator), but for costly-reversible actions at L3+, the system can act within its timeout window.

Target State: Consider implementing an automated disengagement trigger for extreme conditions (sustained high correction rate, failed health checks without operator response within a defined window).

Gap: No fully automated disengagement for scenarios where the operator is unavailable and the system is behaving outside expected parameters.

MANAGE 3: Third-party risk management

MANAGE-3.1 (third-party risk monitoring): Partial. Provider health is monitored (circuit breakers, error rates). Policy and terms-of-service changes are not.

MANAGE-3.2 (pre-trained model monitoring): Partial. Model version changes

are not tracked. A provider could update its model, changing Genesis's behavioral characteristics, without detection. This is a gap.

MANAGE 4: Risk treatment documentation and monitoring

MANAGE-4.1 (post-deployment monitoring): Implemented—Guardian, Sentinel, and correction mechanism provide continuous coverage.

MANAGE-4.2 (continual improvement): Implemented. The learning pipeline (6,175 observations processed), reflection engine, and regular codebase updates constitute ongoing improvement.

MANAGE-4.3: Incidents and errors are communicated to relevant AI actors (Deep Analysis).

Current State: Partial.

Incident communication to the operator is well-implemented. The event bus, Guardian alerts, and Sentinel alarms provide real-time notification of system incidents. Approval request failures, task execution errors, and health threshold violations are all surfaced through Telegram notifications and the dashboard.

Incident communication to affected external parties, on the other hand, is not implemented. If the system sends an inappropriate message, contacts the wrong person, or acts on incorrect information in a way that affects someone outside the system, there is no mechanism to detect that the external party has been affected, notify them, or provide redress. The operator would need to discover the problem through their own review and handle communication manually.

This is a genuine gap for an autonomous agent that takes external actions. The system's monitoring is oriented inward (did the system function correctly?) rather than outward (did the system's actions cause unintended consequences for other people?).

Target State: Implement outreach outcome tracking (did the recipient respond negatively, did a message bounce, did a forum post receive negative feedback). These signals would provide early warning of external impact. Full outward-facing incident communication would require a mechanism for affected parties to contact the operator, which is outside the system's current design scope.

Gap: Significant for external-facing actions. Internal incident communication is well-implemented. External impact detection and communication to affected parties is not.

6. Gap Summary and Prioritization

Critical Gaps

Third-party LLM governance (GOV-6.1): The system's most significant single dependency has the least formal governance. Provider data handling, terms of service monitoring, and model version tracking are all absent. Priority: High.

External impact detection (MANAGE-4.3): The system takes external actions but has no mechanism to detect whether those actions caused unintended harm to recipients. Priority: High for any deployment where outreach is active.

Significant Gaps

Knowledge limits specification (MAP-2.2): The system constrains its authority (what it can do) but not its judgment about competence (what it should attempt). This is a fundamental challenge for LLM-based agents.

Behavioral drift monitoring (MEASURE-2.4): No metric tracks whether system behavior is changing over time below the operator's attention threshold.

Automated disengagement (MANAGE-2.4): No mechanism automatically constrains the system when the operator is unavailable and behavior is outside expected parameters.

Action classification accuracy (MEASURE-1.1): The classification system's false-negative rate (actions misclassified as lower risk than they actually are) is not measured.

Minor Gaps

Periodic review schedule (GOV-1.4/1.5): Review is reactive rather than scheduled. Low risk given the single-operator model but represents a governance best practice gap.

Formal decommissioning procedure (GOV-1.7): Technical shutdown capability exists; procedural checklist does not.

Environmental impact assessment (MEASURE-2.12): Not tracked. Low priority given the system's modest compute footprint.

Framework Adaptation Notes

Several AI RMF subcategories required adaptation for the autonomous agent context:

- **Workforce diversity (GOV-3):** Not applicable to single-operator systems but the underlying concern (diverse perspectives reducing blind spots) is valid and unaddressed.
- **Fairness and bias (MEASURE-2.11):** The RMF's population-level fairness framework does not map to single-user agentic systems. Information prioritization bias and LLM-inherited bias are the relevant concerns.
- **Human oversight (MAP-3.5):** The RMF assumes human review of AI outputs before they affect the world. Genesis operates in a pre-authorized action model where some actions proceed without review. The safety of this model depends on action classification accuracy.
- **TEVV processes (MEASURE-2.1):** Traditional TEVV assumes measurable outputs comparable to ground truth. Genesis produces heterogeneous actions with no single ground truth metric.

These adaptation points represent genuine gaps in the AI RMF's applicability to autonomous agent systems, not failures of this specific implementation. They are discussed further in the methodology document (D4).

7. Remediation Roadmap

The following remediation items are prioritized by risk reduction relative to implementation effort.

Near-term (implementable without architectural changes): 1. Document provider data handling practices and terms of service (addresses GOV-6.1) 2. Establish a quarterly review cadence for steering rules, autonomy levels, and correction patterns (addresses GOV-1.4/1.5) 3. Write a decommissioning procedure (addresses GOV-1.7) 4. Implement correction rate trend tracking for behavioral drift detection (addresses MEASURE-2.4) 5. Define formal evaluation criteria for system performance against intended purposes, beyond implicit operator observation (addresses MANAGE-1.1) 6. Document response procedures for reputational, legal, and regulatory risks that fall outside the technical correction and sentinel mechanisms (addresses MANAGE-2.3)

Medium-term (requires development work): 7. Add outreach outcome tracking to detect negative external impact (addresses MANAGE-4.3) 8. Implement action classification outcome tracking (addresses MEASURE-1.1) 9. Add memory staleness detection for recalled facts (addresses MEASURE-2.4) 10. Implement model version tracking for provider API calls (addresses MANAGE-3.2)

Long-term (requires design decisions): 11. Define explicit knowledge limits and automatic deferral triggers (addresses MAP-2.2) 12. Evaluate automated disengagement triggers for operator-unavailable scenarios (addresses MANAGE-2.4) 13. Develop an agentic-AI-specific bias assessment methodology (addresses MEASURE-2.11)

Appendix A: Operational Evidence

The following data is drawn from the Genesis production database as of 2026-05-06. It is included to ground this assessment in operational reality rather than architectural description.

System Scale

The system has been in continuous operation since 2026-03-10 (58 days at time of assessment). During this period:

- 998 Claude Code sessions have been executed
- 135,843 structured events have been recorded by the event bus
- 12,164 memories have been stored (3,250 in infrastructure, 3,002 in learning, 1,125 in general, 1,049 in channels)

- 6,175 observations have been generated (1,104 awareness ticks, 784 user model updates, 563 light reflections, 494 micro reflections)
- 2,591 cost events have been tracked
- 26 ego cycles have run, producing 28 proposals

Approval Gate Evidence

107 approval requests have been processed through the approval lifecycle:

- 97 approved (90.7%)
- 7 cancelled (6.5%, typically when the triggering alarm resolved before operator response)
- 3 expired (2.8%, operator did not respond within the timeout window)
- 0 rejected by explicit operator decision

All 107 requests were classified as `COSTLY_REVERSIBLE`. No `IRREVERSIBLE` actions have been proposed, which is consistent with the system's deployment context (personal use, no financial transaction capability currently enabled).

Resolution attribution shows the operator engages through multiple channels: 58 via Telegram batch approval (54.2%), 26 via Telegram inline button (24.3%), 8 via dashboard (7.5%), with the remainder via manual administrative actions.

The 0% explicit rejection rate warrants scrutiny. It could indicate that the system's proposals are consistently appropriate, or it could indicate that the operator approves reflexively. The 2.8% expiration rate (3 requests) provides weak counter-evidence: the operator does not approve everything, but the non-approval mode is inaction (timeout) rather than active rejection. This pattern is worth monitoring.

Autonomy State Evidence

Four operational categories are tracked:

- **background_cognitive**: L4 earned, L4 current. 877 successes, 4 corrections. Posterior: 0.998. The high posterior reflects extensive successful operation in this category.
- **direct_session**: L4 earned, L4 current. 5 successes, 1 correction. Posterior: 0.75. Small sample size—the Laplace smoothing dominates at this scale.
- **sub_agent**: L1 earned, L1 current. 0 successes, 0 corrections. No evidence accumulated.
- **outreach**: L1 earned, L1 current. 0 successes, 0 corrections. No evidence accumulated.

The `direct_session` category illustrates a limitation of the Bayesian trust model noted in Section 3 (MAP-2.2): with only 6 total observations, the posterior of 0.75 is sufficient for L4 classification but represents weak statistical evidence. The system reached L4 in this category after approximately 5 successful actions, which is a low bar for the highest autonomy level.

Ego System Evidence

28 proposals generated across 26 cycles: - 10 executed (35.7%) - 11 withdrawn (39.3%, typically superseded by newer proposals or context changes) - 4 tabled (14.3%, deferred for later consideration) - 3 rejected (10.7%)

The 10.7% rejection rate for ego proposals is higher than the 0% rejection rate for approval requests, suggesting that the ego system's proposal quality is lower than the approval gate's action quality. This is expected: ego proposals are more speculative (what should the system do proactively?) while approval requests are more constrained (should the system proceed with a specific identified action?).

Event Distribution

Events by subsystem (top 5): dashboard (74,597), awareness (17,233), surplus (15,600), outreach (12,800), routing (9,772). Events by severity: debug (123,887), error (6,172), warning (4,052), info (1,732). The 4.5% error rate and 3.0% warning rate across 135,843 events represent the system's operational noise floor.

Memory Distribution

12,164 memories across 8 wings: infrastructure (3,250, 26.7%), learning (3,002, 24.7%), unclassified (2,042, 16.8%), general (1,125, 9.2%), channels (1,049, 8.6%), memory (697, 5.7%), autonomy (550, 4.5%), routing (446, 3.7%). The infrastructure and learning wings dominate, reflecting the system's primary operational focus during its first two months.